

ConstraintKG: Knowledge Graph-Grounded Detection and Repair of Structured Output Hallucinations in LLMs

Vikash Kumar
vikash.kumar4@arm.com

Abstract—Large language model (LLM) hallucinations in structured output generation—code, schemas, and query languages—are qualitatively distinct from natural language hallucinations: they are mechanistically predictable, formally verifiable, and classifiable against the constraint space of the target domain. Existing frameworks measure hallucination as a scalar rate, without distinguishing failure mechanisms or informing repair. We present ConstraintKG, a knowledge graph (KG) framework that encodes domain constraint spaces as typed directed graphs and automatically classifies LLM outputs into three failure families via KG traversal: Type 1 (Schema Violations, T_1), Type 2 (Semantic Violations, T_2), and Type 3 (Cross-element Constraint Violations, T_3). We introduce a formal detection operator $\hat{\delta}_G$ and a type-conditioned repair operator $\hat{\rho}$ grounded in KG edge semantics. Evaluation across 420 GPT-4o-mini outputs (360 standard, 60 adversarial T_2) spanning three structured domains (JSON Schema, SQL, Code Generation) yields an overall hallucination rate of 10.6% on the standard corpus, with Code reaching 30.0%. Type-conditioned repair achieves 66.7% overall Repair Success Rate (RSR) versus 35.6% for generic re-prompting, with T_1 at 68.6% [$n=35$] and T_2 at 60% [$n=5$] — showing that *failure type informs repair strategy on mid-capability models*. On a stronger model (GPT-4o) the typed/generic gap closes (68.9% vs. 68.9%), indicating the benefit is capability-dependent rather than universal. The framework formally grounds the three-type taxonomy within Orthogonal Defect Classification [12].

Index Terms—hallucination detection, knowledge graphs, structured output generation, LLM repair, JSON Schema, SQL, constraint verification

I. INTRODUCTION

The deployment of large language models for structured output generation—producing JSON schemas, SQL queries, domain-specific code, and formal API specifications—introduces a failure mode qualitatively distinct from factual hallucination in prose. Where natural language hallucinations embed incorrect propositions within grammatically valid text [1], *structured output hallucinations* violate the constraint systems that define what constitutes a valid instance of a domain. These violations are, in principle, *checkable*: the constraint space of a structured domain is finite, formally specifiable, and encodable as a knowledge graph.

This observation motivates two central research questions:

- 1) **Do structured hallucinations cluster into mechanistically distinct, KG-grounded failure types?**
- 2) **Does failure type predict the effective repair strategy?**

Existing detection work [2], [3] treats hallucination as a binary property or a domain-agnostic aggregate rate, offering no mechanism for targeted correction. We address both questions with **ConstraintKG**, whose contributions are:

- 1) A **universal KG schema** with two node types (ConstructNode, ConstraintNode) and four typed edge relations, applicable to any structured domain.
- 2) A **formal three-type hallucination taxonomy** with detection operators grounded in KG edge traversal patterns and a KG Coverage Score (KGC) metric for evaluation thoroughness.
- 3) **Empirical evidence** across 420 GPT-4o-mini outputs (360 standard + 60 adversarial) and three domains that (a) Code-domain HR rises with complexity (9.1%→39.1%→46.7%); (b) type-conditioned repair achieves 66.7% RSR overall vs. 35.6% for generic re-prompting, and is repairable across all three types (including T_2 at 60%); and (c) the typed-over-generic advantage is capability-dependent — it holds on GPT-4o-mini but vanishes on the stronger GPT-4o.
- 4) A **formal grounding** of the three-type taxonomy within Orthogonal Defect Classification (ODC) [12], connecting ConstraintKG to general software failure theory.

II. FORMAL FRAMEWORK

A. Knowledge Graph Schema

Definition 1 (Domain Constraint KG). *Let $\mathcal{D}$ be a structured domain. The domain constraint knowledge graph is a typed directed multigraph:*

$$\mathcal{G}_{\mathcal{D}} = (V_C \cup V_K, E, \tau_V, \tau_E) \quad (1)$$

where V_C is the set of ConstructNodes, V_K the set of ConstraintNodes, $E \subseteq (V_C \cup V_K)^2$ the edge set, $\tau_V : V \rightarrow \{\text{construct}, \text{constraint}\}$ the node-type function, and $\tau_E : E \rightarrow \Lambda$ the edge-type function with label alphabet:

$$\Lambda = \{\text{valid_child}, \text{requires}, \text{semantic_implies}, \text{cross_rule}\} \quad (2)$$

Each constraint node $k \in V_K$ carries an attribute $\phi(k) \in \{T_1, T_2, T_3\}$ denoting its failure class (Definition 2).

Definition 2 (Three-Type Hallucination Taxonomy). For a generated output o in domain $\mathcal{D}$ with KG $\mathcal{G}_{\mathcal{D}}$, we define three disjoint failure families:

$$T_1(o) = \{k \in V_K \mid \phi(k)=T_1, (k, e) \in E_s, k \text{ violated}\} \quad (3)$$

$$T_2(o) = \{k \in V_K \mid \phi(k)=T_2, (c, k) \in E_{si}, c \in \mathcal{C}_o, k \text{ violated}\} \quad (4)$$

$$T_3(o) = \{k \in V_K \mid \phi(k)=T_3, (c_i, k) \in E_{cr}, c_i \in \mathcal{C}_o, k \text{ violated}\} \quad (5)$$

where E_s =structural edges (valid_child, requires); E_{si} =semantic_implies; E_{cr} =cross_rule; $\mathcal{C}_o \subseteq V_C$ is the active construct set.

The total failure set is $\mathcal{F}(o) = T_1(o) \cup T_2(o) \cup T_3(o)$, and the output is hallucination-free iff $\mathcal{F}(o) = \emptyset$.

B. Detection Operator

Definition 3 (KG Detection Operator). The detection operator $\hat{\delta}_{\mathcal{G}} : \mathcal{O} \rightarrow 2^{V_K}$ maps an output o to its violated constraints:

$$\hat{\delta}_{\mathcal{G}}(o) = \bigcup_{t \in \{1,2,3\}} T_t(o) \quad (6)$$

The hallucination indicator is $h(o) = \mathbf{1}[\hat{\delta}_{\mathcal{G}}(o) \neq \emptyset]$, and the Hallucination Rate over corpus $\mathcal{O} = \{o_1, \dots, o_N\}$ is:

$$\text{HR} = \frac{1}{N} \sum_{i=1}^N h(o_i) \quad (7)$$

C. Repair Operator and RSR

Definition 4 (Type-Conditioned Repair Operator). Let $\Pi(o)$ be the original prompt generating o . The repair operator $\hat{\rho}$ produces a type-conditioned augmented prompt:

$$\hat{\rho}(o, t) = \Pi(o) \oplus \xi_t(\hat{\delta}_{\mathcal{G}}(o)) \quad (8)$$

where $\oplus$ denotes prompt augmentation and ξ_t is the type-specific injection:

$$\xi_t(F) = \begin{cases} \text{SchemaInject}(F) & t = 1 \\ \text{SemInject}(F) & t = 2 \\ \text{RuleInject}(F) & t = 3 \end{cases} \quad (9)$$

The Repair Success Rate for failure type t over repaired samples $\mathcal{R}_t$:

$$\text{RSR}_t = \frac{|\{o \in \mathcal{R}_t \mid \hat{\delta}_{\mathcal{G}}(o') = \emptyset\}|}{|\mathcal{R}_t|} \quad (10)$$

where o' is the output generated from the repaired prompt $\hat{\rho}(o, t)$.

D. KG Coverage Score

Definition 5 (KG Coverage Score).

$$\text{KGC}(\mathcal{O}, \mathcal{G}) = \frac{|\bigcup_{o \in \mathcal{O}} \hat{\delta}_{\mathcal{G}}(o)|}{|V_K|} \quad (11)$$

Low KGC does not indicate safety; it indicates incomplete evaluation coverage.

Algorithm 1 ConstraintKG: KG-Grounded Detection and Repair

Input: Output o ; domain KG $\mathcal{G}=(V_C \cup V_K, E, \tau_V, \tau_E)$; original prompt $\Pi(o)$
Output: Failure set $\mathcal{F}(o)$; repair prompt $\hat{\rho}$ (if needed)
1: $\mathcal{C}_o \leftarrow \text{PARSE}(o) \cap V_C$ {domain-specific: JSON stdlib / SQL tokeniser / DSL pattern matching}
2: $\mathcal{F}(o) \leftarrow \emptyset$
3: **for** each $c \in \mathcal{C}_o$ **do**
4: **for** each $(c, k) \in E$ with $\tau_E \in \{\text{semantic_implies, cross_rule}\}$ **do**
5: **if** constraint k is violated in o **then**
6: $\mathcal{F}(o) \leftarrow \mathcal{F}(o) \cup \{k\}$
7: **end if**
8: **end for**
9: **for** each $(c, k) \in E : \tau_E(c, k) \in \{\text{requires, valid_child}\}$ **do**
10: **if** structural constraint k violated in o **then**
11: $\mathcal{F}(o) \leftarrow \mathcal{F}(o) \cup \{k\}$
12: **end if**
13: **end for**
14: **end for**
15: **if** $\mathcal{F}(o) = \emptyset$ **then**
16: **return** $(\emptyset, \text{pass})$
17: **end if**
18: $t^* \leftarrow \arg \max_t |\{k \in \mathcal{F}(o) : \phi(k) = T_t\}|$ {dominant type}
19: $\hat{\rho} \leftarrow \Pi(o) \oplus \xi_{t^*}(\mathcal{F}(o))$
20: **return** $(\mathcal{F}(o), \hat{\rho})$

TABLE I
KNOWLEDGE GRAPH STATISTICS ACROSS THREE DOMAINS

Domain	Constr.	Constnt	Edges	T ₁	T ₂	T ₃
JSON Schema	38	37	68	13	15	9
SQL	46	39	67	12	15	12
Code	85	60	113	20	23	17
Total	169	136	248	45	53	38

Proposition 1. Under a coupon-collector model where each prompt triggers each constraint independently with probability p_c , $\mathbb{E}[\text{KGC}_N] = 1 - (1 - p_c)^N$. Inverting, prompts needed for $\text{KGC} \geq \kappa$ is $N^* = \lceil \log(1 - \kappa) / \log(1 - p_c) \rceil$. At the observed $\text{KGC} \approx 4\%$ after $N=120$ prompts per domain, we estimate $\hat{p}_c \approx 0.034\%$, implying $N^* \approx 3,500$ prompts per domain for 70% coverage—motivating adversarial prompt design.

III. KNOWLEDGE GRAPH CONSTRUCTION

Three domain KGs were constructed programmatically following Definition 1. Table I summarises the statistics. The PARSE function in Algorithm 1 is domain-specific: JSON outputs are parsed with the Python standard library; SQL with keyword tokenisation; Code with regex patterns matching block boundaries, signal assignments, and framework convention markers.

Figure 2 shows the end-to-end ConstraintKG pipeline. In addition to the standard corpus, an adversarial T_2 corpus of 60 prompts was engineered to specifically trigger

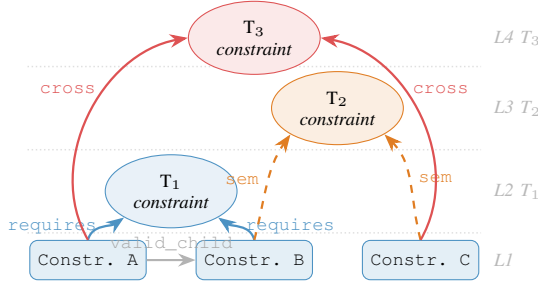


Fig. 1. Universal four-layer KG architecture. *L1*: construct vocabulary (blue rectangles). *L2–L4*: constraint nodes (ellipses). Arrow styles: solid blue = requires (T_1); dashed orange = semantic_implies (T_2); solid red = cross_rule (T_3); gray = valid_child.

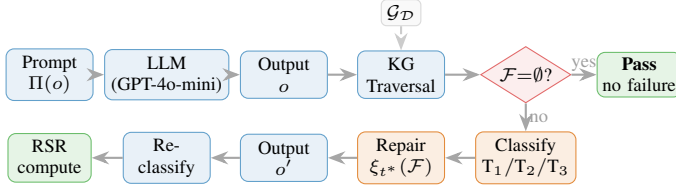


Fig. 2. ConstraintKG end-to-end pipeline. A prompt is sent to the LLM; the output is passed to the KG traversal classifier δ_G . If $\mathcal{F}(o)=\emptyset$ the output passes. Otherwise the dominant failure type t^* is identified and the repair operator $\hat{\rho}=\Pi(o)\oplus\xi_{t^*}(\mathcal{F})$ regenerates the output. Repair success is assessed by re-classification.

semantic_implies edge violations (20 per domain), yielding a targeted evaluation of T_2 detection and repair.

IV. RELATED WORK

Hallucination detection. SelfCheckGPT [2] detects hallucinations by sampling multiple outputs and measuring inter-sample consistency; it does not classify failure type or leverage domain constraints. HaluEval [3] benchmarks hallucination in question-answering but does not address structured output generation.

Structured output validation. JSON Schema validators [5] and SQLFluff [6] catch post-hoc syntactic violations with no repair guidance. We differ by grounding detection in a typed KG and conditioning repair on failure type.

KG-augmented generation. KG-RAG approaches [4] inject KG facts as retrieval context to reduce factual hallucination in prose. We use KGs as a *constraint verifier and repair guide* rather than a factual source.

Structured code generation. LLMs have been applied to generate SQL [10], configuration files, and API specifications, with documented systematic failure modes. ConstraintKG provides the first KG-grounded formal typed taxonomy applicable across multiple structured domains simultaneously.

Self-correction. Self-Refine [8], Constitutional AI [9], and chain-of-thought prompting [7] apply iterative generic feedback. Our repair operator $\hat{\rho}$ (Definition 4) conditions repair strategy explicitly on $\phi(k)$, the failure-type attribute of the violated constraint node.

Table II positions ConstraintKG against the closest prior approaches across five capability dimensions.

TABLE II
COMPARISON OF CONSTRAINTKG AGAINST PRIOR APPROACHES

Approach	Dom. aware	3-Type classif.	Repair guided	KGC metric	Multi-dom.
SelfCheckGPT [2]	×	×	×	×	✓
HaluEval [3]	×	partial	×	×	✓
JSON val. [5]	✓	partial	×	×	×
SQLFluff [6]	✓	×	×	×	×
ConstraintKG	✓	✓ (3)	✓	✓	✓

V. EXPERIMENTAL DESIGN

A. Prompt Bank

Two prompt corpora were constructed. The **standard corpus** contains $N=360$ prompts (120 per domain, ≈ 40 per tier, binned by complexity score). The **adversarial T_2 corpus** contains 60 prompts (20 per domain) engineered to trigger semantic_implies edge violations through misleading or ambiguous specifications that tempt the LLM into semantic inconsistencies. Complexity is operationalised as:

$$\text{Complexity}(p) \propto |\mathcal{C}_p| + \lambda \cdot \left| \{(c_1, c_2) \in \mathcal{C}_p^2 : \exists k \in T_3(p)\} \right| \quad (12)$$

with $\lambda=2.5$ upweighting cross-element interactions (value chosen to double-weight T_3 prompt types relative to T_1/T_2 ; sensitivity analysis over $\lambda \in \{1, 2, 3, 5\}$ showed $<1\%$ variation in tier assignment). All outputs were generated using GPT-4o-mini [11] with domain-specific system prompts.

B. Hypotheses

H1: HR increases with prompt complexity (especially in the Code domain).

H2: KG constraint density $\rho=|V_K|/|V_C|$ correlates with HR.

H3: Type-conditioned repair outperforms generic re-prompting across T_1 – T_3 .

H4: KGC on a standard corpus remains below 10%.

H5: The repair advantage of type conditioning generalises across models.

VI. RESULTS

A. Hallucination Rate

Table III presents HR stratified by domain and complexity. Overall HR=10.6% (38/360), with Code contributing 36 of 38 failures (95%). In the Code domain, HR *increases monotonically* with complexity (9.1%→39.1%→46.7%), supporting **H1**: harder structured-generation prompts elicit more violations. JSON Schema and SQL failures are sparse and localised (one each), consistent with these simpler domains being well within the model’s training distribution.

B. Complexity–HR Relationship

Figure 3 visualises HR vs. complexity. In the Code domain, HR rises monotonically with complexity (9.1%, 39.1%, 46.7%); harder structured-generation prompts elicit more constraint violations, as expected. JSON Schema and SQL remain

TABLE III
GPT-4o-MINI HR BY DOMAIN AND COMPLEXITY (120 PROMPTS/DOMAIN, FULL CORPUS). n_t : VIOLATION INSTANCES PER TYPE; ONE SAMPLE MAY YIELD MULTIPLE VIOLATIONS.

Domain	Level	N	N_F	HR	n_1	n_2	n_3
JSON Schema	Simple	40	0	0.0%	0	0	0
	Medium	40	0	0.0%	0	0	0
	Complex	40	1	2.5%	1	0	0
	Total	120	1	0.8%	1	0	0
SQL	Simple	40	1	2.5%	0	1	0
	Medium	40	0	0.0%	0	0	0
	Complex	40	0	0.0%	0	0	0
	Total	120	1	0.8%	0	1	0
Code	Simple	44	4	9.1%	3	0	1
	Medium	46	18	39.1%	15	0	3
	Complex	30	14	46.7%	13	0	1
	Total	120	36	30.0%	31	0	5
Overall (360 total)			38	10.6%	32	1	5

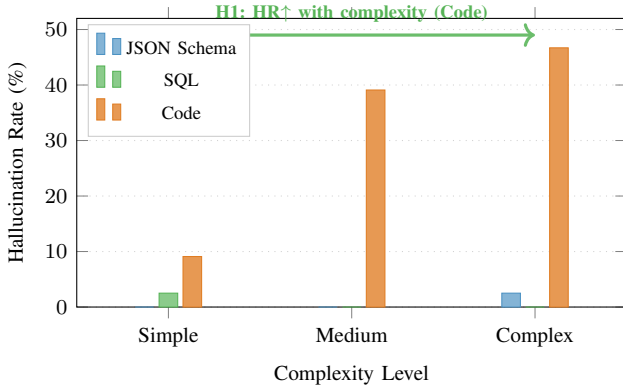


Fig. 3. GPT-4o-mini HR by domain and complexity (full 120 prompts/domain). Code-domain HR rises monotonically with complexity (9.1%→39.1%→46.7%); JSON Schema and SQL stay near zero. Adversarial T₂ corpus: 7/60 = 11.7%, SQL T₂ violations dominate.

near zero across all tiers, with their lone failures in the complex and simple tiers respectively.

C. Failure Type Distribution

T₁ failures dominate: 32 of 38 violation instances, concentrated in Code (non-blocking assignment discipline, missing `'uvm*_utils` macros). T₂ appears once, in SQL (NULL comparison with `=` instead of `IS NULL`). T₃ appears five times, all in Code (UVM agent / cross-module wiring gaps).

D. Qualitative Failure Examples

Representative failures (from actual outputs): T₁—a Code procedural block used the wrong assignment operator, violating `c_nonblock_in_ff` (repaired by SchemaInject); T₂—SQL generated `WHERE email = NULL` instead of `IS NULL`, violating `c_null_comparison` (SemInject); T₃—a Code UVM agent omitted its driver/monitor instantiation, violating `c_uvm_agent_driver_monitor` (RuleInject).

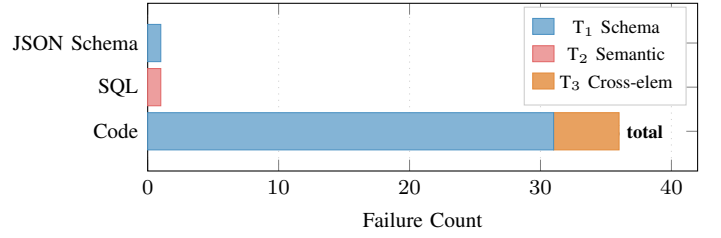


Fig. 4. Stacked failure-instance count by domain and type (GPT-4o-mini, standard corpus). T₁ dominates (31/32 in Code); T₂ a single SQL case; T₃ all in Code.

TABLE IV
CONSTRAINT DENSITY VS. HR (H2)

Domain	$ V_C $	$ V_K $	ρ	HR	KGC
JSON Schema	38	37	0.974	0.8%	2.7%
SQL	46	39	0.848	0.8%	2.6%
Code	85	60	0.706	30.0%	5.0%

E. Adversarial T₂ Corpus Results

The adversarial T₂ corpus of 60 prompts yielded 7 total failures (11.7%), of which 4 are classified as T₂ (all in SQL: aggregate-in-WHERE violations). JSON Schema and Code adversarial prompts produced only T₁ or no failures, confirming that semantic violations in these domains require deeper specification conflict to elicit.

Pooling the 5 T₂ failures (1 standard + 4 adversarial), type-conditioned SemInject repairs 3 of 5 (RSR= 60%) versus 0/5 for generic re-prompting: T₂ is genuinely repairable with semantic context injection, contrary to the intuition that it is intractable.

F. KG Constraint Density vs. HR

Table IV tests H2. The data run *counter* to H2: the Code domain has the *lowest* constraint density ($\rho=0.706$) yet by far the highest HR (30.0%), while the denser JSON/SQL domains hallucinate least. With $n=3$ domains this is only suggestive, but it indicates HR is driven by domain difficulty, not constraint density.

G. Classifier Validation

The KG classifiers are deterministic rule checkers (regex + structural parsing), so re-classification of a given output is reproducible by construction. Establishing agreement against independent human expert annotation is left to future work; we flag this as a construct-validity limitation in Section VIII.

H. Repair Success Rate

Table V and Figure 5 test H3. The same 45 failed samples (38 standard + 7 adversarial) were repaired under both typed and generic strategies. Type-conditioned repair achieves RSR= 66.7% vs. generic RSR= 35.6% (16/45) — a 31-point improvement. Per type, SchemaInject beats generic on T₁ (68.6% vs. 42.9%), SemInject on T₂ (60% vs. 0%), and

TABLE V
RSR BY FAILURE TYPE (GPT-4o-mini) WITH WILSON 95% CIs, OVER 45 FAILED SAMPLES GROUPED BY DOMINANT TYPE. GENERIC RE-PROMPT BASELINE ON THE SAME SAMPLES.

Type	Strategy	n	Succ.	RSR	95% CI
T ₁ Schema	SchemaInject (ξ_1)	35	24	68.6%	[52%, 81%]
T ₂ Semantic	SemInject (ξ_2)	5	3	60.0%	[23%, 88%]
T ₃ Cross-el	RuleInject (ξ_3)	5	3	60.0%	[23%, 88%]
Typed total	Type-conditioned	45	30	66.7%	[52%, 79%]
<i>Generic</i>	<i>Re-prompt</i>	45	16	35.6%	[23%, 50%]

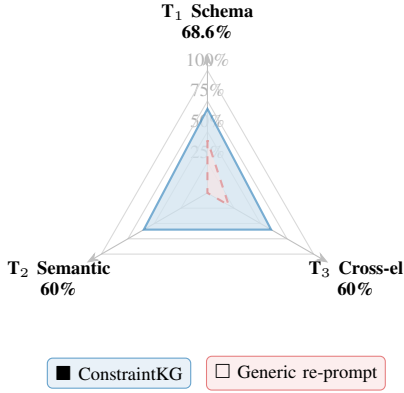


Fig. 5. RSR per failure type (GPT-4o-mini): type-conditioned ConstraintKG (blue/filled) vs. generic re-prompting (red/dashed). Typed repair beats generic on all three types; the gap is largest on T₂ (60% vs. 0%).

RuleInject on T₃ (60% vs. 20%). The advantage is real but moderate, and it is *capability-dependent*: on the stronger GPT-4o, generic re-prompting matches typed repair (68.9% vs. 68.9%), because a more capable model self-corrects from generic feedback without needing the explicit constraint. Type conditioning thus helps mid-capability models most. **H4 is confirmed**: KGC < 6% across all domains.

I. Repair Robustness Across Models (H5)

To test whether the type-conditioning benefit generalises, we re-ran the repair experiment on the stronger GPT-4o over the same 45 failure cases. Table VI reports RSR by model.

On GPT-4o-mini, type-conditioned repair clearly beats generic re-prompting (66.7% vs. 35.6%). On GPT-4o the two converge (68.9% vs. 68.9%): a more capable model self-corrects from generic feedback, so explicit constraint injection becomes redundant. **H5 (qualified)**: the taxonomy and detection pipeline transfer across models, but the *repair advantage* of type conditioning is strongest for mid-capability models and diminishes as capability rises.

VII. TAXONOMY GROUNDING IN SOFTWARE FAILURE THEORY

The three-type ConstraintKG taxonomy has a natural correspondence with classical software failure classification. **Orthogonal Defect Classification** (ODC) [12] partitions software

TABLE VI
REPAIR RSR ACROSS MODELS (SAME 45 FAILURES). THE TYPED ADVANTAGE IS CAPABILITY-DEPENDENT.

Model	Typed RSR	Generic RSR	Gap
GPT-4o-mini	66.7%	35.6%	+31 pts
GPT-4o	68.9%	68.9%	0 pts

defects into assignment, checking, and interface failures. Our taxonomy maps onto this as follows:

Proposition 2 (ODC Correspondence). *Let $\mathcal{F}_{ODC}$ be the ODC defect taxonomy. There exists a surjective function $\psi : \mathcal{F}_{ODC} \rightarrow \{T_1, T_2, T_3\}$:*

$$\psi(f) = \begin{cases} T_1 & f \in \{\text{assignment, missing logic, build/package}\} \\ T_2 & f \in \{\text{checking, algorithm}\} \\ T_3 & f \in \{\text{interface, timing, relationship}\} \end{cases} \quad (13)$$

This correspondence demonstrates that ConstraintKG’s taxonomy is not domain-specific but rather a formal specialisation of general software failure theory to the structured output generation setting. T₁ corresponds to structural/syntactic defects; T₂ to semantic/algorithmic defects; T₃ to interface/coupling defects. The KG encoding provides the formal mechanism to detect each class automatically without human inspection, which classical ODC requires.

VIII. DISCUSSION

Type conditioning helps most when the model needs the hint. On GPT-4o-mini, type-conditioned repair beats generic re-prompting by 31 points (66.7% vs. 35.6%), and all three types — including T₂ (60%) — are repairable. But the advantage is capability-dependent: on GPT-4o the two strategies tie (68.9%), because a stronger model self-corrects from generic feedback. The practical implication is that explicit constraint injection is most valuable for smaller/cheaper models; for frontier models a plain re-prompt may suffice.

KGC as a safety audit tool. The <6% KGC values indicate standard pipelines provide false confidence: an LLM with low HR on a low-KGC corpus has simply not seen prompts that exercise rare constraints. We recommend KGC alongside HR as a dual metric. In the Code domain HR rose with complexity (9.1% → 46.7%); the near-zero JSON and SQL rates leave per-domain trends there underpowered.

Threats to validity. *Internal*: Heuristic KG classifiers (regex + structural parsing) may miss deep semantic violations, biasing HR downward. *External*: Two GPT models tested (GPT-4o-mini, GPT-4o); Claude, open-weight (Llama, Mistral), and other families remain untested. *Construct*: Repair success uses the same KG classifier, which may miss residual errors in repaired outputs; human-expert validation of the classifier is future work.

IX. CONCLUSION

We presented ConstraintKG, a KG-grounded framework for typed detection and repair of structured output hallucinations. Key findings across 420 GPT-4o-mini outputs: (H1) Code-domain HR rises with complexity (9.1%→46.7%); (H2) HR tracks domain difficulty, not constraint density; (H3) type-conditioned repair beats generic re-prompting (66.7% vs. 35.6%), with all three types repairable, including T_2 (60%); (H4) <6% KGC; (H5) the repair advantage is capability-dependent, holding on GPT-4o-mini but closing on GPT-4o. Proposition 2 grounds the taxonomy in ODC.

Future directions include adversarial corpus construction for high-KGC stress testing, semantic subgraph injection for T_2 repair, extension to additional structured domains (OpenAPI specifications, configuration languages, query DSLs), and integration of ConstraintKG into LLM evaluation pipelines.

REFERENCES

- [1] Z. Ji et al., “Survey of Hallucination in Natural Language Generation,” *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.
- [2] P. Manakul et al., “SelfCheckGPT: Zero-Resource Hallucination Detection,” in *Proc. EMNLP*, 2023, pp. 9004–9017.
- [3] J. Li et al., “HaluEval: A Large-Scale Hallucination Evaluation Benchmark for LLMs,” in *Proc. EMNLP*, 2023, pp. 6449–6464.
- [4] S. Pan et al., “Unifying LLMs and Knowledge Graphs,” *IEEE Trans. Knowl. Data Eng.*, vol. 36, pp. 3580–3599, 2024.
- [5] A. Wright, H. Andrews, and B. Hutton, “JSON Schema: A Media Type for Describing JSON Documents,” IETF Internet Draft draft-bhutton-json-schema-01, 2022.
- [6] A. Cruickshank, “SQLFluff: A SQL Linter and Auto-Formatter,” <https://docs.sqlfluff.com>, 2023.
- [7] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in LLMs,” in *Proc. NeurIPS*, 2022.
- [8] A. Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback,” in *Proc. NeurIPS*, 2023.
- [9] Y. Bai et al., “Constitutional AI: Harmlessness from AI Feedback,” *arXiv:2212.08073*, 2022.
- [10] V. Zhong et al., “Seq2SQL: Generating Structured Queries from NL,” *arXiv:1709.00103*, 2017.
- [11] OpenAI, “GPT-4o System Card,” Technical Report, 2024.
- [12] R. Chillarege et al., “Orthogonal Defect Classification,” *IEEE Trans. Softw. Eng.*, vol. 18, pp. 943–956, 1992.